



DATA DRIVEN GROWTH WITH PYTHON

Know Your Metrics

Learn what and how to track with Python



Barış Karaman

Follow

8 min read · May 4, 2019



1.7K



20



Introduction

This series of articles was designed to explain how to use Python in a simplistic way to fuel your company's growth by applying the predictive approach to all your actions. It will be a combination of programming, data analysis, and machine learning.

I will cover all the topics in the following nine articles:

1- **Know Your Metrics**

2- Customer Segmentation

3- Customer Lifetime Value Prediction

4- Churn Prediction

5- Predicting Next Purchase Day

6- Predicting Sales

7- Market Response Models

8- Uplift Modeling

9- A/B Testing Design and Execution

Articles will have their own code snippets to make you easily apply them. If you are super new to programming, you can have a good introduction for Python and Pandas (a famous library that we will use on everything) here.

But still without a coding introduction, you can learn the concepts, how to use your data and start generating value out of it:

Sometimes you gotta run before you can walk —
Tony Stark

As a pre-requisite, be sure Jupyter Notebook and Python are installed on your computer. The code snippets will run on Jupyter Notebook only.

Alright, let's start.

Part 1: Know Your Metrics



We all remember Captain Sparrow's famous compass that shows the location of what he wants the most. Without a **North Star Metric**, this is how we are in terms of growth. We want more customers, more orders, more revenue, more signups, more efficiency...

Before going into coding, we need to understand what exactly is North Star Metric. If you already know and track it, this post can help you do a deep

dive analysis with Python. If you don't know, first you should find yours (probably you are already tracking it but didn't name it as North Star conceptually). This is how **Sean Ellis** describes it:

The North Star Metric is the single metric that best captures the core value that your product delivers to customers.

This metric depends on your company's product, position, targets & more. Airbnb's North Star Metric is nights booked whereas for Facebook, it is daily active users.

In our example, we will be using a sample dataset of an online retail. For an online retail, we can select our North Star Metric as **Monthly Revenue**. Let's see how our data look like on jupyter notebook.

Monthly Revenue

Let's start with importing the libraries we need and reading our data from CSV with the help of pandas:

Open in app ↗

Sign up

Sign in

Medium



Search



Write



```

1 # import libraries
2 from datetime import datetime, timedelta
3 import pandas as pd
4 %matplotlib inline
5 import matplotlib.pyplot as plt
6 import numpy as np
7 import seaborn as sns
8 from __future__ import division
9
10 import plotly.plotly as py
11 import plotly.offline as pyoff
12 import plotly.graph_objs as go
13
14 #initiate visualization library for jupyter notebook
15 pyoff.init_notebook_mode()
16
17 tx_data = pd.read_csv('data.csv')
18
19 tx_data.head(10)

```

g1_load_data.py hosted with ❤ by GitHub

[view raw](#)

This is how our data looks like:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
5	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8:26	7.65	17850.0	United Kingdom
6	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/2010 8:26	4.25	17850.0	United Kingdom
7	536366	22633	HAND WARMER UNION JACK	6	12/1/2010 8:28	1.85	17850.0	United Kingdom
8	536366	22632	HAND WARMER RED POLKA DOT	6	12/1/2010 8:28	1.85	17850.0	United Kingdom
9	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	12/1/2010 8:34	1.69	13047.0	United Kingdom

We have all the crucial information we need:

- Customer ID

- Unit Price
- Quantity
- Invoice Date

With all these features, we can build our North Star Metric equation:

$$\text{Revenue} = \text{Active Customer Count} * \text{Order Count} * \text{Average Revenue per Order}$$

It's time to get our hands dirty. We want to see monthly revenue but unfortunately there is no free lunch. Let's engineer our data:

```
1 #converting the type of Invoice Date Field from string to datetime.
2 tx_data['InvoiceDate'] = pd.to_datetime(tx_data['InvoiceDate'])
3
4 #creating YearMonth field for the ease of reporting and visualization
5 tx_data['InvoiceYearMonth'] = tx_data['InvoiceDate'].map(lambda date: 100*date.y
6
7 #calculate Revenue for each row and create a new dataframe with YearMonth - Rev
8 tx_data['Revenue'] = tx_data['UnitPrice'] * tx_data['Quantity']
9 tx_revenue = tx_data.groupby(['InvoiceYearMonth'])['Revenue'].sum().reset_index(
10 tx_revenue
```

g1_monthly_revenue.py hosted with ❤ by GitHub

[view raw](#)

Good job, now we have a dataframe that shows our monthly revenue:

tx_revenue		
	InvoiceYearMonth	Revenue
0	201012	748957.020
1	201101	560000.260
2	201102	498062.650
3	201103	683267.080
4	201104	493207.121
5	201105	723333.510
6	201106	691123.120
7	201107	681300.111
8	201108	682680.510
9	201109	1019687.622
10	201110	1070704.670
11	201111	1461756.250
12	201112	433668.010

Next step, visualization. A line graph would be sufficient:

```

1  #X and Y axis inputs for Plotly graph. We use Scatter for line graphs
2  plot_data = [
3      go.Scatter(
4          x=tx_revenue['InvoiceYearMonth'],
5          y=tx_revenue['Revenue'],
6      )
7  ]
8
9  plot_layout = go.Layout(
10     xaxis={"type": "category"},
11     title='Montly Revenue'
12 )
13 fig = go.Figure(data=plot_data, layout=plot_layout)
14 pyoff.iplot(fig)

```

g1_visual_revenue.py hosted with ❤ by GitHub

[view raw](#)

Jupyter notebook output:



This clearly shows our revenue is growing especially Aug '11 onwards (and our data in December is incomplete). Absolute numbers are fine, let's figure out what is our **Monthly Revenue Growth Rate**:

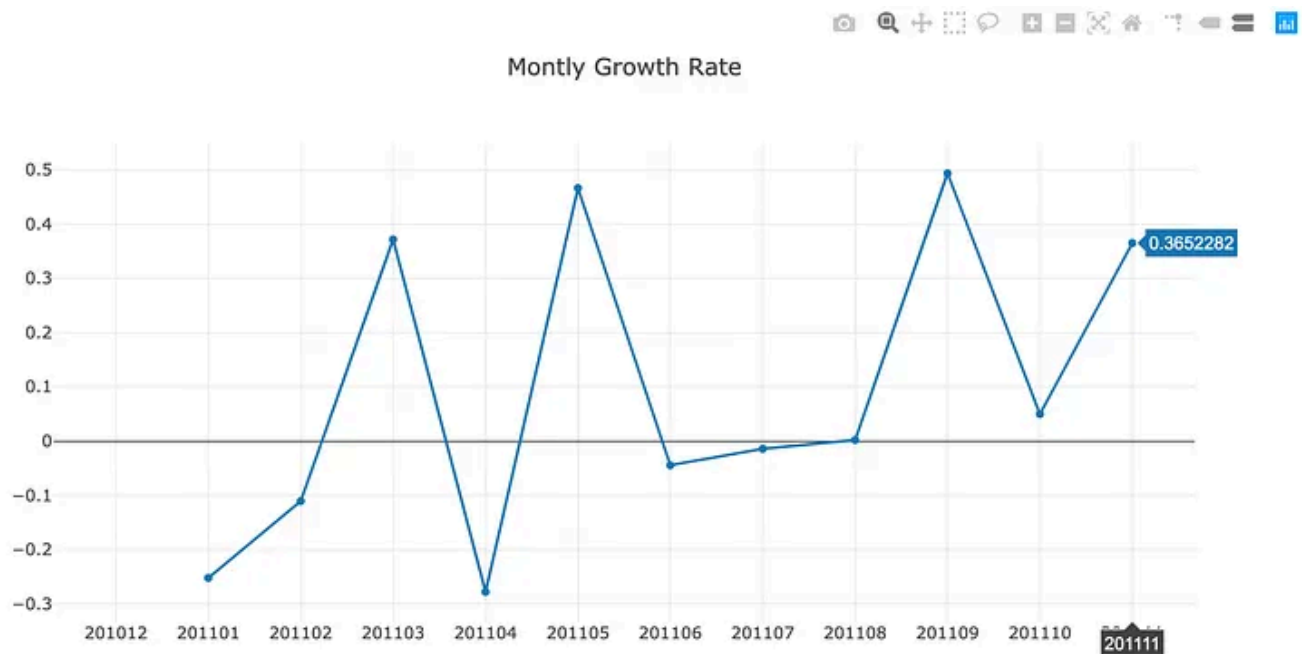

```

1  #using pct_change() function to see monthly percentage change
2  tx_revenue['MonthlyGrowth'] = tx_revenue['Revenue'].pct_change()
3
4  #showing first 5 rows
5  tx_revenue.head()
6
7  #visualization - line graph
8  plot_data = [
9      go.Scatter(
10         x=tx_revenue.query("InvoiceYearMonth < 201112")['InvoiceYearMonth'],
11         y=tx_revenue.query("InvoiceYearMonth < 201112")['MonthlyGrowth'],
12     )
13 ]
14
15  plot_layout = go.Layout(
16      xaxis={"type": "category"},
17      title='Montly Growth Rate'
18  )
19
20  fig = go.Figure(data=plot_data, layout=plot_layout)
21  pyoff.iplot(fig)

```

g1_m_revenue_growth.py hosted with ♥ by GitHub

[view raw](#)



Everything looks good, we saw 36.5% growth previous month (December is excluded in the code since it hasn't been completed yet). But we need to identify what exactly happened on April. Was it due to less active customers or our customers did less orders? Maybe they just started to buy cheaper products? We can't say anything without doing a deep-dive analysis.

Monthly Active Customers

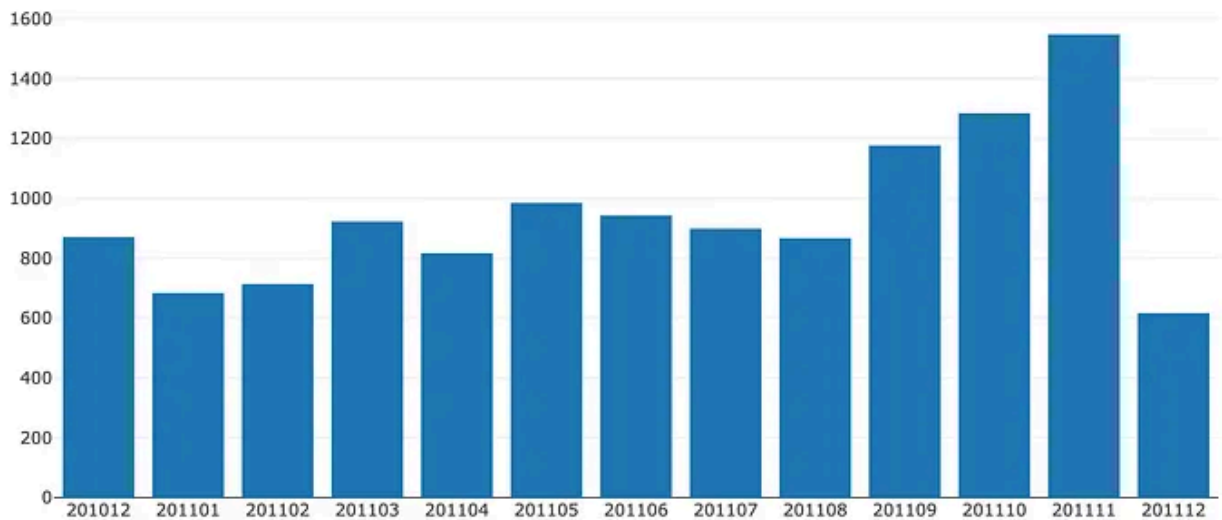
To see the details Monthly Active Customers, we will follow the steps we exactly did for Monthly Revenue. Starting from this part, we will be focusing on UK data only (which has the most records). We can get the monthly active customers by counting unique *CustomerIDs*. Code snippet and the output are as follows:

No. of active customers per month and its bar plot:

tx_monthly_active

	InvoiceYearMonth	CustomerID
0	201012	871
1	201101	684
2	201102	714
3	201103	923
4	201104	817
5	201105	985
6	201106	943
7	201107	899
8	201108	867
9	201109	1177
10	201110	1285
11	201111	1548
12	201112	617

Monthly Active Customers



In April, Monthly Active Customer number dropped to 817 from 923 (-11.5%).

Get Barış Karaman's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

We will see the same trend for number of orders as well.

Monthly Order Count

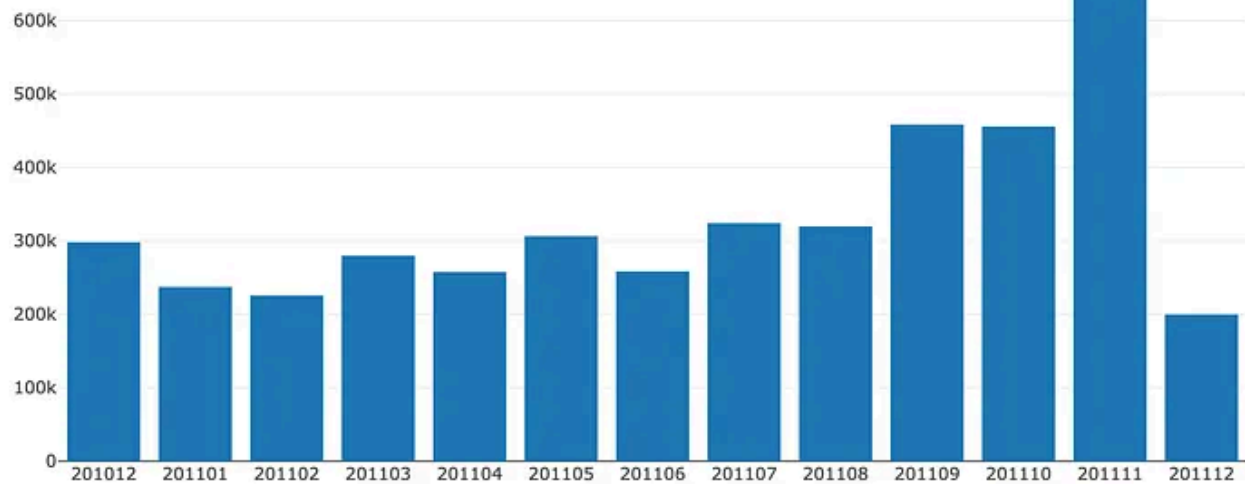
We will apply the same code by using *Quantity* field:

Monthly order count and its bar plot:

tx_monthly_sales

	InvoiceYearMonth	Quantity
0	201012	298101
1	201101	237381
2	201102	225641
3	201103	279843
4	201104	257666
5	201105	306452
6	201106	258522
7	201107	324129
8	201108	319804
9	201109	458490
10	201110	455612
11	201111	642281
12	201112	199907

Monthly Total # of Order



As we expected, Order Count is also declined in April (279k to 257k, -8%)

We know that Active Customer Count directly affected Order Count decrease. At the end, we should definitely check our **Average Revenue per Order** as well.

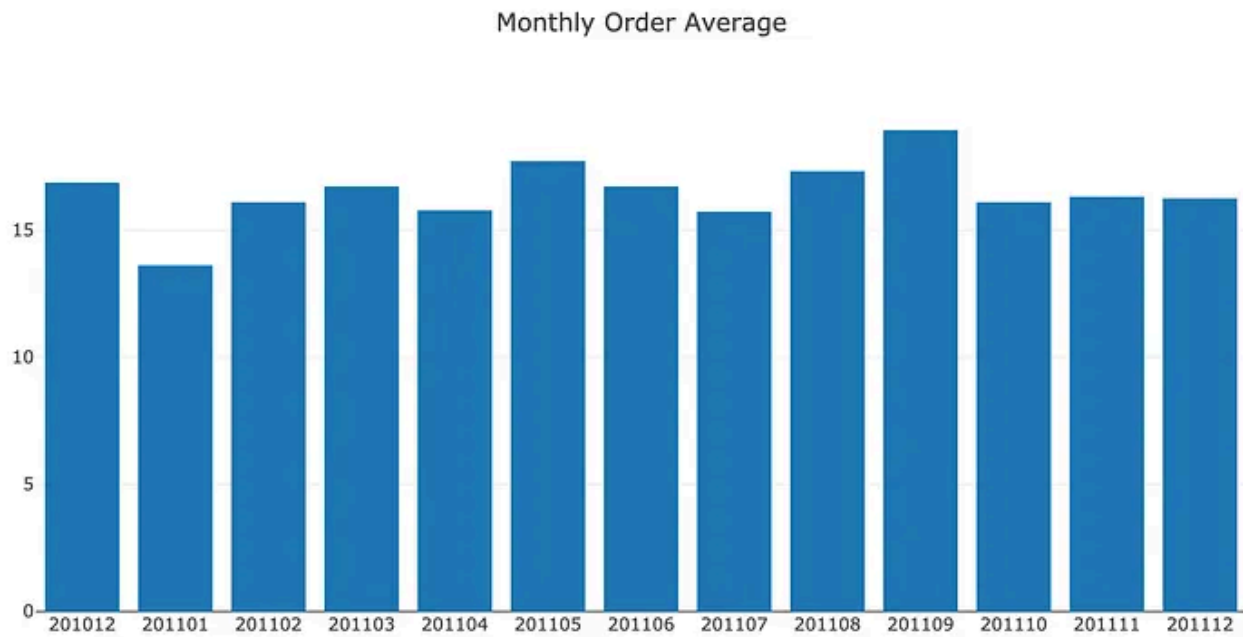
Average Revenue per Order

To get this data, we need to calculate the average of revenue for each month:

Monthly average revenue per order and its bar plot:

tx_monthly_order_avg

	InvoiceYearMonth	Revenue
0	201012	16.865860
1	201101	13.614680
2	201102	16.093027
3	201103	16.716166
4	201104	15.773380
5	201105	17.713823
6	201106	16.714748
7	201107	15.723497
8	201108	17.315899
9	201109	18.931723
10	201110	16.093582
11	201111	16.312383
12	201112	16.247406



Even the monthly order average dropped for April (16.7 to 15.8). We observed slow-down in every metric affecting our North Star.

We have looked at our major metrics. Of course there are many more and it varies across industries. Let's continue investigating some other important metrics:

- **New Customer Ratio:** a good indicator of if we are losing our existing customers or unable to attract new ones
- **Retention Rate:** King of the metrics. Indicates how many customers we retain over specific time window. We will be showing examples for monthly retention rate and cohort based retention rate.

New Customer Ratio

First we should define what is a new customer. In our dataset, we can assume a new customer is whoever did his/her first purchase in the time window we defined. We will do it monthly for this example.

We will be using `.min()` function to find our first purchase date for each customer and define new customers based on that. The code below will apply this function and show us the revenue breakdown for each group monthly.

Dataframe output after merging with First Purchase Date:

```
tx_uk.head()
```

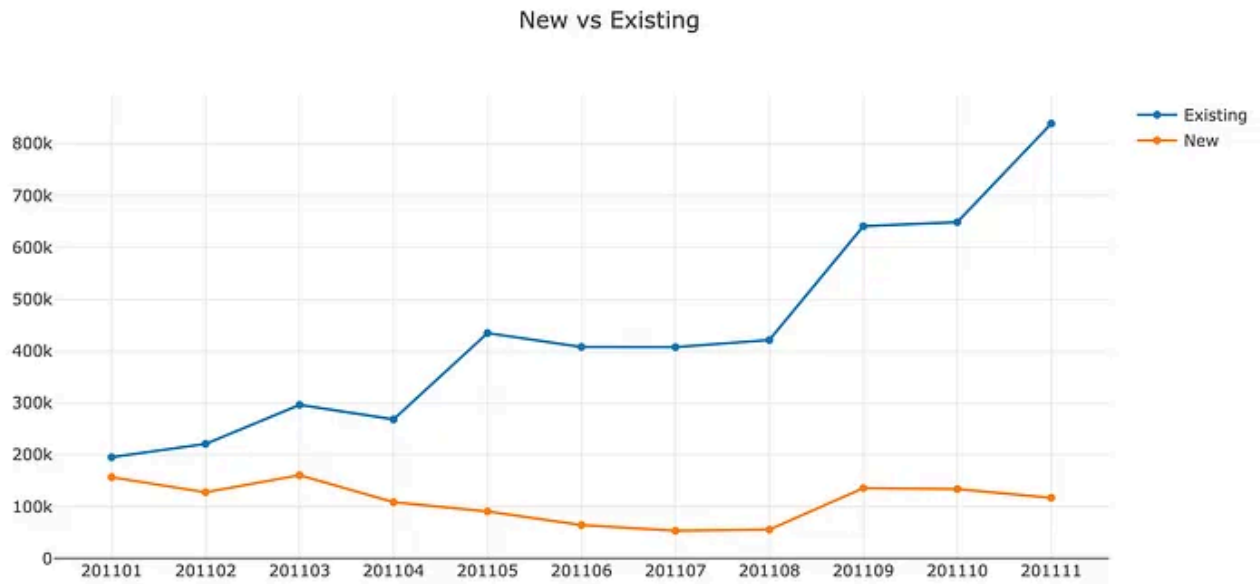
StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	InvoiceYearMonth	Revenue	MinPurchaseDate	MinPurchaseYearMonth	UserType
85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom	201012	15.30	2010-12-01 08:26:00	201012	New
71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom	201012	20.34	2010-12-01 08:26:00	201012	New
84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom	201012	22.00	2010-12-01 08:26:00	201012	New
84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom	201012	20.34	2010-12-01 08:26:00	201012	New
84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom	201012	20.34	2010-12-01 08:26:00	201012	New

Revenue per month for New and Existing Customers:

```
tx_user_type_revenue.query("InvoiceYearMonth != 201012 and InvoiceYearMonth != 201112")
```

	InvoiceYearMonth	UserType	Revenue
1	201101	Existing	195275.510
2	201101	New	156705.770
3	201102	Existing	220994.630
4	201102	New	127859.000
5	201103	Existing	296350.030
6	201103	New	160567.840
7	201104	Existing	268226.660
8	201104	New	108517.751
9	201105	Existing	434725.860
10	201105	New	90847.490
11	201106	Existing	408030.060
12	201106	New	64479.190
13	201107	Existing	407693.610
14	201107	New	53453.991
15	201108	Existing	421388.930
16	201108	New	55619.480
17	201109	Existing	640861.901
18	201109	New	135667.941
19	201110	Existing	648837.600
20	201110	New	133940.280
21	201111	Existing	838955.910
22	201111	New	117153.750

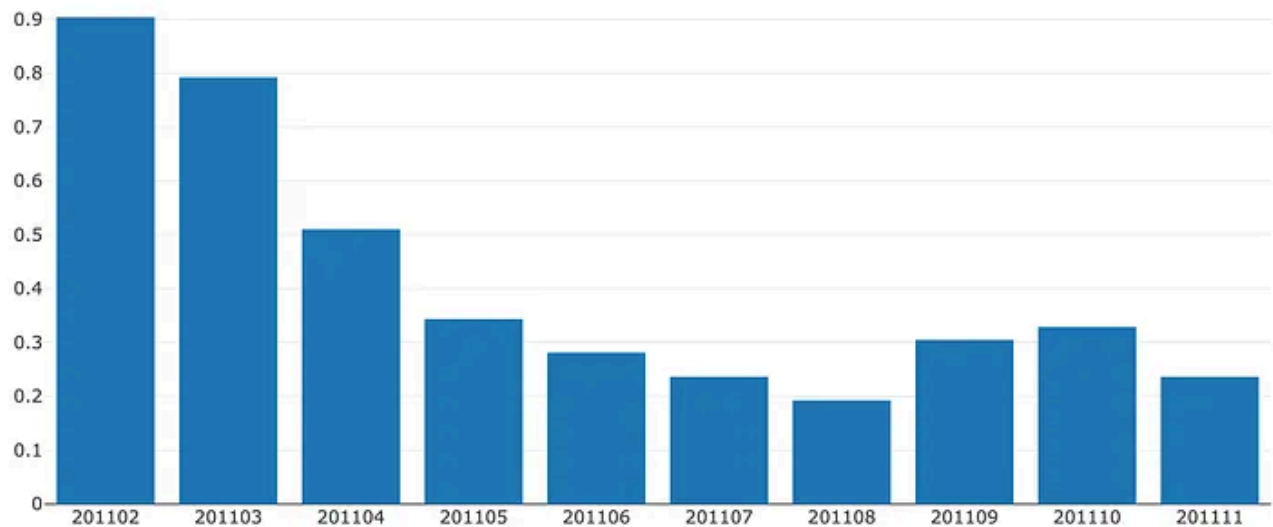
Line chart of the above:



Existing customers are showing a positive trend and tell us that our customer base is growing but new customers have a slight negative trend.

Let's have a better view by looking at the New Customer Ratio:

New Customer Ratio



New Customer Ratio has declined as expected (we assumed on Feb, all customers were New) and running around 20%.

Monthly Retention Rate

Retention rate should be monitored very closely because it indicates how sticky is your service and how well your product fits the market. For making Monthly Retention Rate visualized, we need to calculate how many customers retained from previous month.

$$\text{Monthly Retention Rate} = \frac{\text{Retained Customers From Prev. Month}}{\text{Active Customers Total}}$$

We will be using `crosstab()` function of pandas which makes calculating Retention Rate super easy.

First, we create a dataframe that shows total monthly revenue for each customer:

```
tx_user_purchase = tx_uk.groupby(['CustomerID', 'InvoiceYearMonth'])['Revenue'].sum().astype(int).reset_index()
```

```
tx_user_purchase
```

	CustomerID	InvoiceYearMonth	Revenue
0	12346.0	201101	0
1	12747.0	201012	706
2	12747.0	201101	303
3	12747.0	201103	310
4	12747.0	201105	771
5	12747.0	201106	376
6	12747.0	201108	301
7	12747.0	201110	675
8	12747.0	201111	312
9	12747.0	201112	438
10	12748.0	201012	4177
11	12748.0	201101	418
12	12748.0	201102	389

`crosstab()` function converts it to retention table:

```
tx_retention = pd.crosstab(tx_user_purchase['CustomerID'], tx_user_purchase['InvoiceYearMonth']).reset_index()
```

```
tx_retention.head()
```

	InvoiceYearMonth	CustomerID	201012	201101	201102	201103	201104	201105	201106	201107	201108	201109	201110	201111	201112
0	12346.0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	12747.0	1	1	1	0	1	0	1	1	0	1	0	1	1	1
2	12748.0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
3	12749.0	0	0	0	0	0	0	1	0	0	1	0	0	1	1
4	12820.0	0	1	0	0	0	0	0	0	0	0	1	1	0	1

Retention table shows us which customers are active on each month (1 stands for active).

With the help of a simple *for loop*, for each month we calculate Retained Customer Count from previous month and Total Customer Count.

In the end, we have our Retention Rate dataframe & line chart like below:

```
: tx_retention['RetentionRate'] = tx_retention['RetainedUserCount']/tx_retention['TotalUserCount']
```

```
: tx_retention
```

	InvoiceYearMonth	RetainedUserCount	TotalUserCount	RetentionRate
0	201102	263	714	0.368347
1	201103	305	923	0.330444
2	201104	310	817	0.379437
3	201105	369	985	0.374619
4	201106	417	943	0.442206
5	201107	379	899	0.421580
6	201108	391	867	0.450980
7	201109	417	1177	0.354291
8	201110	502	1285	0.390661
9	201111	616	1548	0.397933
10	201112	402	617	0.651540

Monthly Retention Rate



Monthly Retention Rate significantly jumped from June to August and went back to previous levels afterwards.

Cohort Based Retention Rate

There is another way of measuring Retention Rate which allows you to see Retention Rate for each cohort. Cohorts are determined as first purchase

year-month of the customers. We will be measuring what percentage of the customers retained after their first purchase in each month. This view will help us to see how recent and old cohorts differ regarding retention rate and if recent changes in customer experience affected new customer's retention or not.

This will be a bit more complicated than others in terms of coding.

Tx_retention has this amazing view of cohort based retention rate:

	TotalUserCount	201101	201102	201103	201104	201105	201106	201107	201108	201109	201110	201111	201112
InvoiceYearMonth													
201101	362	1.0	0.23	0.28	0.25	0.34	0.29	0.26	0.26	0.31	0.35	0.36	0.15
201102	339	NaN	1.00	0.25	0.19	0.28	0.28	0.25	0.26	0.28	0.28	0.31	0.10
201103	408	NaN	NaN	1.00	0.19	0.26	0.22	0.23	0.17	0.26	0.24	0.29	0.09
201104	276	NaN	NaN	NaN	1.00	0.22	0.22	0.22	0.21	0.23	0.23	0.26	0.08
201105	252	NaN	NaN	NaN	NaN	1.00	0.23	0.17	0.17	0.21	0.24	0.27	0.10
201106	207	NaN	NaN	NaN	NaN	NaN	1.00	0.21	0.16	0.25	0.26	0.32	0.10
201107	172	NaN	NaN	NaN	NaN	NaN	NaN	1.00	0.20	0.19	0.23	0.28	0.11
201108	140	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.00	0.26	0.23	0.26	0.14
201109	275	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.00	0.29	0.33	0.12
201110	318	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.00	0.27	0.13
201111	296	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.00	0.14
201112	34	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.00

We can see that first month retention rate became better recently (don't take Dec '11 into account) and in almost 1 year, 15% of our customers retain with us.

Finally...We know our metrics and how to track/analyze them with Python.

You can find the jupyter notebook of this article [here](#).

Let's try to segment our base to see who are our best customers in [Part 2](#).

I've started to write the more advanced and updated version of my articles [here](#). Feel free to visit, learn more and support.

Data Science

Machine Learning

Python

Data Driven Growth

Growth



Published in TDS Archive

Follow

828K followers · Last published Feb 3, 2025

An archive of data science, data analytics, data engineering, machine learning, and artificial intelligence writing from the former Towards Data Science Medium publication.



Written by Barış Karaman

Follow

3.5K followers · 301 following

Growth Hacker <https://www.linkedin.com/in/karamanbaris> Schedule a free call with me here: <https://app.growthmentor.com/baris-karaman>
